

IAW · UNIDAD 4

PHP detrás de Nginx

El camino de una petición, de ida y vuelta

HOY

Tres cosas, nada más

1 

Hacer que PHP funcione detrás de Nginx

Cinco líneas de configuración

2 

Saber qué tramo falla cuando algo se rompe

404, 502, 504 y dónde mirar

3 

Cerrar un fallo de seguridad

Un formulario que se deja engañar

EMPEZAMOS POR EL PROBLEMA

Pedimos la página… y nos llevamos el código

en el cliente: docker compose exec cliente sh

```
$ curl -kI https://club.iaw.local/
Content-Type: application/octet-stream        ← «un fichero cualquiera»

$ curl -k https://club.iaw.local/
<?php                                         ← ¡el código fuente!
$titulo = 'Inicio · Club de Lectura';
```

En el navegador: https://localhost:8445

El fichero index.php se DESCARGA. Ábrelo: es el código, tal cual.

GRAVE — Dentro de un .php puede estar la contraseña de la base de datos.

POR QUÉ PASA

Nginx hace lo único que sabe: entregar ficheros

index.html

Ya es la página. Nginx la coge del disco y la entrega. Todos ven lo mismo: web estática.

index.php

No es la página: es un programa que la fabrica en cada visita. Hay que ejecutarlo: web dinámica.

Un .html se entrega. Un .php se ejecuta. Y Nginx no sabe ejecutar PHP.

LO USAN — WordPress, Moodle y Nextcloud son PHP: los veréis este curso.

EJECUTAR

El .php es el programa; el HTML, su resultado

index.php · en el servidor

```php
<p>Son las <?= date('H:i') ?></p>

<?php foreach ($libros as $l): ?>
  <li><?= $l ?></li>
<?php endforeach; ?>
```

lo que recibe el navegador

```html
<p>Son las 18:05</p>

  <li>El camino</li>
  <li>Nada</li>
  <li>Luces de bohemia</li>
```

se ejecuta

se entrega

El código se queda en el servidor. Al navegador solo le llega HTML.

El camino de una petición a PHP



En la demo faltaba el paso 2. Nginx no sabía que los .php van a PHP-FPM, así que entregó el fichero. Los .css y las imágenes sí se quedan en Nginx: no gastan PHP.

TODA LA UNIDAD — Lo que veamos a partir de ahora pasa en un tramo de este dibujo. Míralo arriba.

POR SI TE AYUDA

Lo mismo, dicho con un restaurante



Cliente

el navegador

Camarero

Nginx

Cocina

PHP-FPM

El camarero no cocina. El .php es la receta; el HTML, el plato.

DESDE AQUÍ — Volvemos a los nombres de verdad: Nginx, PHP-FPM y worker.

PASO 2

Cinco líneas que enseñan a Nginx el camino



nginx/sites-available/club.conf

```
location ~ \.php$ {
  try_files $uri =404;
  fastcgi_pass  php:9000;
  include       fastcgi_params;
  fastcgi_param SCRIPT_FILENAME
    $document_root$fastcgi_script_name;
}
```

Si acaba en .php…
`location ~ \.php$`

…y el fichero existe…
`try_files`

…pásaselo a PHP-FPM…
`fastcgi_pass php:9000`

…y dile qué fichero ejecutar.
`SCRIPT_FILENAME`

En castellano: «lo que acabe en .php, a PHP-FPM».

NO SE MEMORIZA — Se entiende. La regla está en los apuntes y en el kit.

DEMO

Editar, comprobar, recargar… y el mismo curl de antes



1 Edita
quita las # del bloque
`club.conf`

2 Comprueba
¿está bien escrito?
`nginx -t`

3 Recarga
sin cortar el servicio
`nginx -s reload`

en el cliente: el mismo curl

```
$ curl -kI https://club.iaw.local/
Content-Type: text/html; charset=UTF-8        ← ¡una página!
X-Powered-By: PHP/8.3.x                       ← (lo quitamos en la sesión 2)
$ curl -k https://club.iaw.local/
<h2>Lecturas del trimestre (4)</h2>           ← HTML, sin rastro de PHP
```

SI nginx -t FALLA — No recargues. Lee el error: dice la línea que está mal.

CUANDO ALGO FALLA

Cada error señala un punto del camino



404

Not Found (de nginx)

En Nginx

El fichero no existe: PHP ni se entera

Mira:

`club_access.log`

404

«File not found.»

En PHP

PHP no sabe qué fichero ejecutar: revisa SCRIPT_FILENAME

Mira:

`docker compose logs php`

502

Bad Gateway

Tramo 2

No conecta con PHP: puerto mal o PHP apagado

Mira:

`club_error.log`

504

Gateway Timeout

En PHP

PHP no contesta a tiempo: más de 60 s

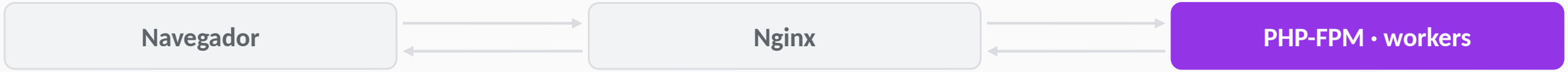
Mira:

`club_error.log`

EL MÉTODO — Primero el código (curl -kI), después el log de Nginx y por último el de PHP.

LO QUE VIAJA POR EL CAMINO

No vamos a programar: solo a leer PHP

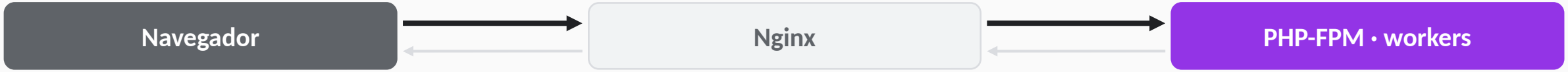


`<?php … ?>`	Aquí empieza y acaba el PHP. Lo de fuera es HTML normal
`$libros`	Una variable: un valor con nombre. Siempre con $
`<?= $titulo ?>`	Imprime: pon aquí el valor de la variable
`foreach`	Repite: una vez por cada libro

PRUÉBALO — Cambia un libro en www/club/inc/libros.php y recarga la página: cambia al momento.

PASO 1

Lo que escribe el usuario viaja en la petición



navegador

```
https://localhost:8445/recomendar.php?nombre=Ana&libro=Nada
```

recomendar.php

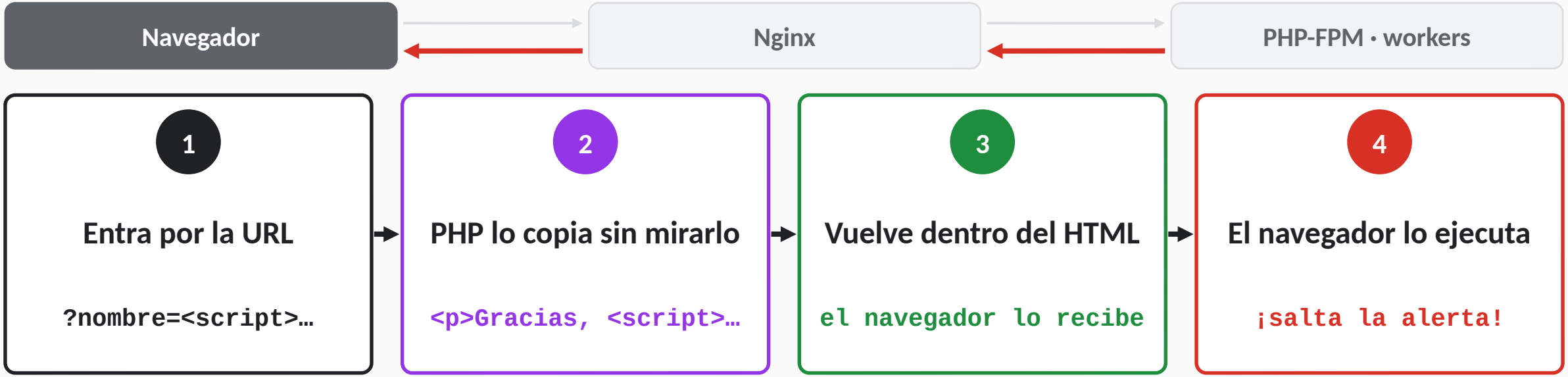
```php
$nombre = $_GET['nombre'];          // vale "Ana"
<p>Gracias, <?= $nombre ?>.</p>
```

Lo que va detrás de ? lo escribe el usuario. Puede escribir CUALQUIER COSA.

NUNCA — Contraseñas en la URL: se quedan en el historial y en los logs.

EL FALLO DE SEGURIDAD

XSS: el camino lleva código hasta el navegador

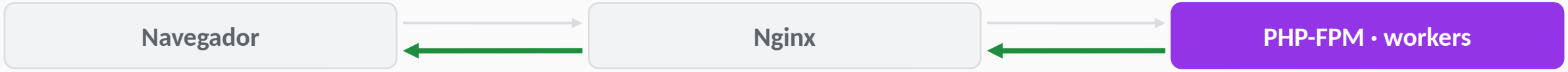


No se ejecuta en el servidor: se ejecuta en el navegador de quien abre el enlace.

SE LLAMA — XSS (Cross-Site Scripting). Con otro código, podría robar la sesión de esa persona.

EL ARREGLO

Una función, justo antes de imprimir: htmlspecialchars



ANTES ✘

```
<p>Gracias, <?= $nombre ?>.</p>
```

DESPUÉS ✔

```
<p>Gracias,
<?= htmlspecialchars($nombre) ?>.</p>
```

Llega <script>

El navegador lo ejecuta

Llega <script>

El navegador lo enseña como texto

Lo que escribió el usuario, antes de imprimirlo: htmlspecialchars.

REGLA DE ORO — Nunca te fíes de lo que escribe el usuario.

TU TURNO

Cinco pasos, en parejas

1	Arranca el laboratorio	`docker compose up -d --build`
2	Quita las # de la regla	`nginx/sites-available/club.conf`
3	Comprueba y recarga	`nginx -t` y `nginx -s reload`
4	Arregla el formulario	`htmlspecialchars` en `www/club/recomendar.php`
5	Comprueba	`bash comprobar.sh` → 4 de 10

SI ACABAS ANTES — Borra la línea SCRIPT_FILENAME y recarga: ¿qué sale? ¿En qué punto del camino falla?

LO QUE NOS LLEVAMOS HOY

Resumen de puntos clave

- Navegador → Nginx → PHP-FPM → Nginx → navegador
- Un .html se entrega; un .php se ejecuta
- Sin location ~ \.php$, se descarga el código
- Cada error señala un punto: 404 · 502 · 504
- Lo que escribe el usuario: htmlspecialchars al imprimir

LA PRÓXIMA SESIÓN

PHP listo para un servidor de verdad

- Que no enseñe sus errores
- Que no diga su versión
- ¿Cuántos workers caben?

Nginx recibe, PHP-FPM ejecuta y al navegador solo le llega HTML.



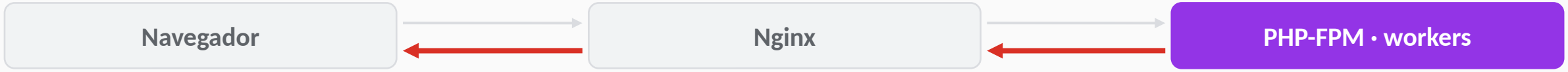
IAW · UNIDAD 4 · SESIÓN 2

El mismo camino, en producción

1. ¿Qué no debe salir hacia el navegador?
2. ¿Cuántas peticiones caben a la vez en PHP-FPM?
3. ¿Qué petición tarda, y por qué?

PHP.INI

Seis líneas: que PHP cuente menos y pueda menos



php/zz-lab.ini

```
expose_php          = Off
display_errors      = Off
log_errors          = On
error_log           = /var/log/lab/php-errores.log
disable_functions   = exec,shell_exec,system,…
upload_max_filesize = 4M
```

No dice su versión
Sin la cabecera X-Powered-By

No enseña sus errores
Van a un log, no al navegador

No ejecuta órdenes del sistema
Aunque se cuele un .php malicioso

Limita las subidas
4 MB como mucho

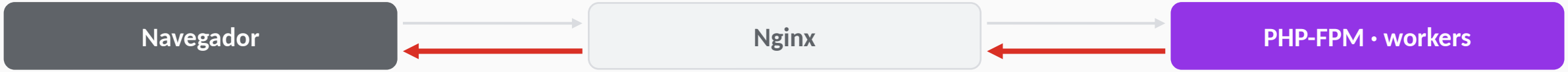
aplicar y comprobar

```
$ docker compose restart php
$ docker compose exec cliente curl -kI https://club.iaw.local/
```

SI NO CAMBIA NADA — PHP-FPM lee el php.ini al arrancar: sin restart, sigue con la configuración vieja.

ERRORES

El error, para el log; no para el navegador



navegador · display_errors = On ✘

```
Warning: require(/var/www/club/inc/
  configuracion-que-no-existe.php):
  Failed to open stream …
Fatal error: Uncaught Error: Failed
  opening required … in
  /var/www/club/error.php:4
```

el navegador ve tus rutas

logs/php-errores.log · display_errors = Off ✔

```
PHP Fatal error:  Uncaught Error:
  Failed opening required
  '/var/www/club/inc/…'
  in /var/www/club/error.php:4

(el usuario solo ve un 500)
```

solo tú lo lees

En producción: display_errors = Off y log_errors = On. Siempre.

EN DESARROLLO — En tu ordenador sí puedes ver los errores (display_errors = On). En el servidor, nunca.

DENTRO DE PHP-FPM

Cada worker atiende una petición a la vez

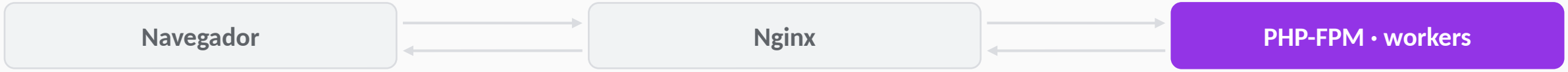


pm	Cuántos workers	Cuándo
`dynamic`	Entre un mínimo y un máximo	Lo normal
`static`	Siempre max_children	Mucho tráfico constante
`ondemand`	Los crea al llegar peticiones	Poco tráfico, poca memoria

EL NÚMERO QUE IMPORTA — pm.max_children. De fábrica es 5, y hay que calcularlo.

EL CÁLCULO

¿Cuántos workers caben? Se mide y se calcula




```
docker compose exec php ps -o pid,rss,cmd -C php-fpm     (RSS en KB, bajo carga)

  1 24060 php-fpm: master process
 13 12824 php-fpm: pool www                  ← un worker: ≈ 12 MB con esta web tan pequeña
```

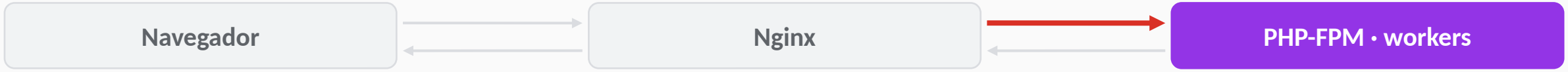
Memoria	Reserva	Por worker	= 10
512 MB	− 112 MB	÷ 40 MB	workers
mem_limit del compose	el maestro y margen	aplicación real: 30–60 MB	

pm.max_children = 10

¿POR QUÉ 40 Y NO 12? — Esta web es mínima. En un servidor real se mide con la aplicación real, bajo carga.

SATURACIÓN

Más peticiones que workers: cola



2 workers

6 peticiones a la vez → 8 segundos

10 workers

Las mismas 6 peticiones → 4 segundos

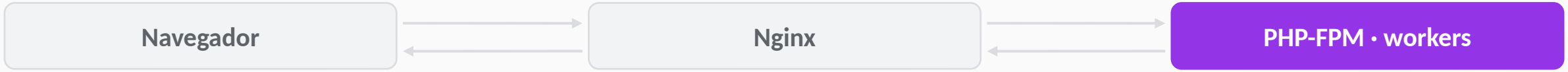
```
docker compose logs php
WARNING: [pool www] server reached pm.max_children setting (2), consider raising it
```

Si la cola dura demasiado → 504, aunque el servidor esté tranquilo.

FÍJATE — Un 504 no siempre es «el servidor va lento»: a veces faltan workers.

RENDIMIENTO

slowlog: en qué línea se atasca un worker



php/zz-lab-pool.conf

```
request_slowlog_timeout = 3s
slowlog = /var/log/lab/php-lento.log
```

se configura en el pool

logs/php-lento.log

```
script_filename = /var/www/club/lento.php
[0x…] sleep() /var/www/club/lento.php:5
```

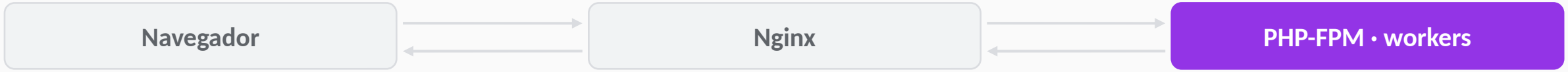
la línea exacta

No adivines qué va lento: el slowlog te dice el fichero y la línea.

EN DOCKER — El slowlog necesita cap_add: [SYS_PTRACE]. En el laboratorio ya está puesto.

RENDIMIENTO

OPcache: no compilar el mismo .php cada vez



Sin OPcache

Cada petición lee y compila el fichero otra vez

Con OPcache

Compilado una vez y guardado en memoria

php.ini

```
opcache.enable = 1
opcache.validate_timestamps = 1   ; 1: mira si el .php ha cambiado · 0: no lo mira (producción)
```

TRAMPA — Con validate_timestamps = 0, si cambias un .php sigue saliendo lo viejo: reinicia PHP-FPM.

TU TURNO

Del 4 al 10 de 10, en parejas

1	**php.ini**	php/zz-lab.ini: las seis líneas + restart php
2	**Errores**	error.php: ¿qué ve el navegador? ¿Qué hay en el log?
3	**Pool**	Mide el RSS, calcula pm.max_children, escribe tu cuenta
4	**Cierra**	info.php e inc/ fuera: deny all en club.conf
5	**Comprueba**	bash comprobar.sh → 10 de 10

SI ACABAS ANTES — Pon pm.max_children = 2 y lanza el ab de la demo: ¿cuánto tarda? ¿Qué dice el log?

LO QUE NOS LLEVAMOS DE LA UD4

Resumen de puntos clave

- expose_php = Off: PHP no dice su versión
- display_errors = Off: los errores, al log
- Cada worker atiende una petición a la vez
- pm.max_children = memoria libre ÷ memoria de un worker
- slowlog: la línea que tarda · OPcache: no recompilar

LA UD5

El mismo camino, con Python

- PHP-FPM → Gunicorn
- fastcgi_pass → proxy_pass
- Una API que devuelve JSON

Producción: que salga solo lo que debe, con los workers que caben.